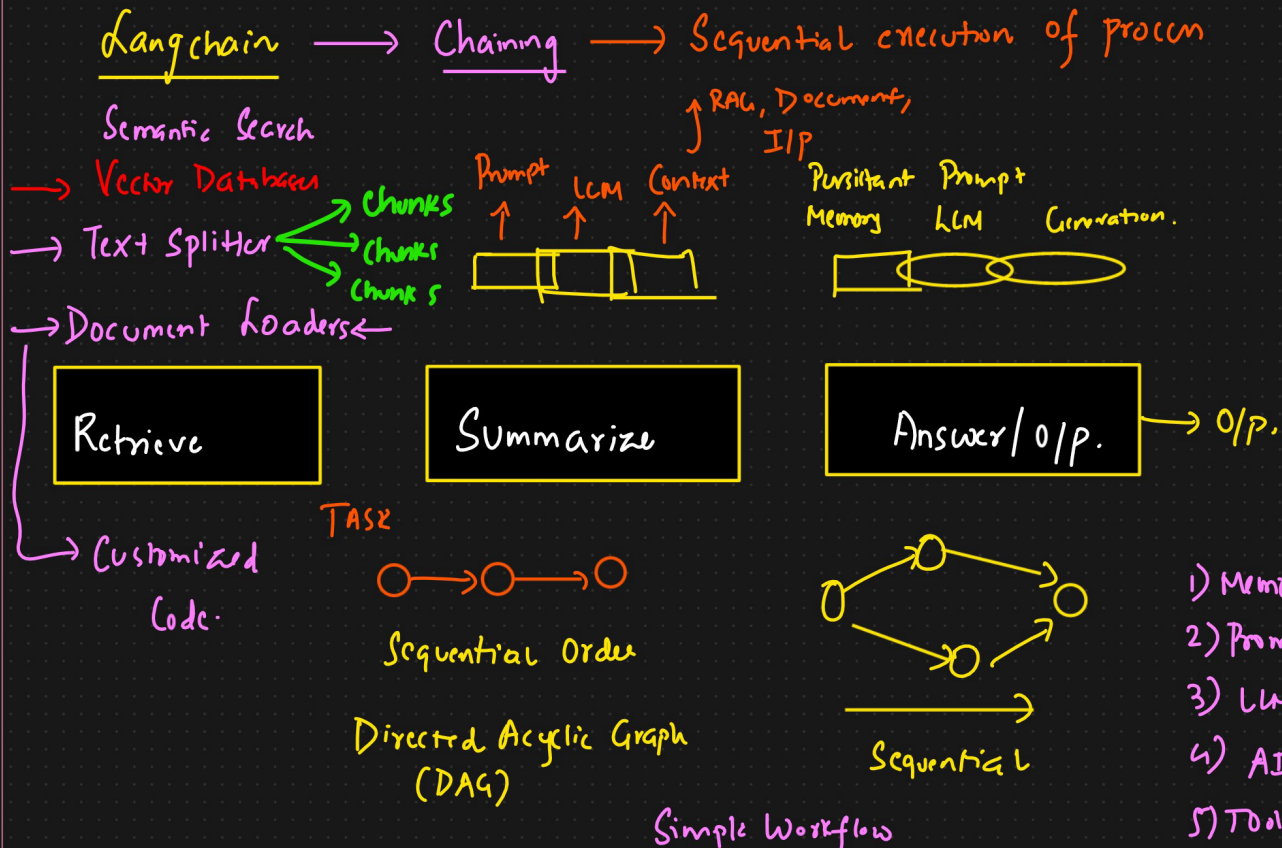
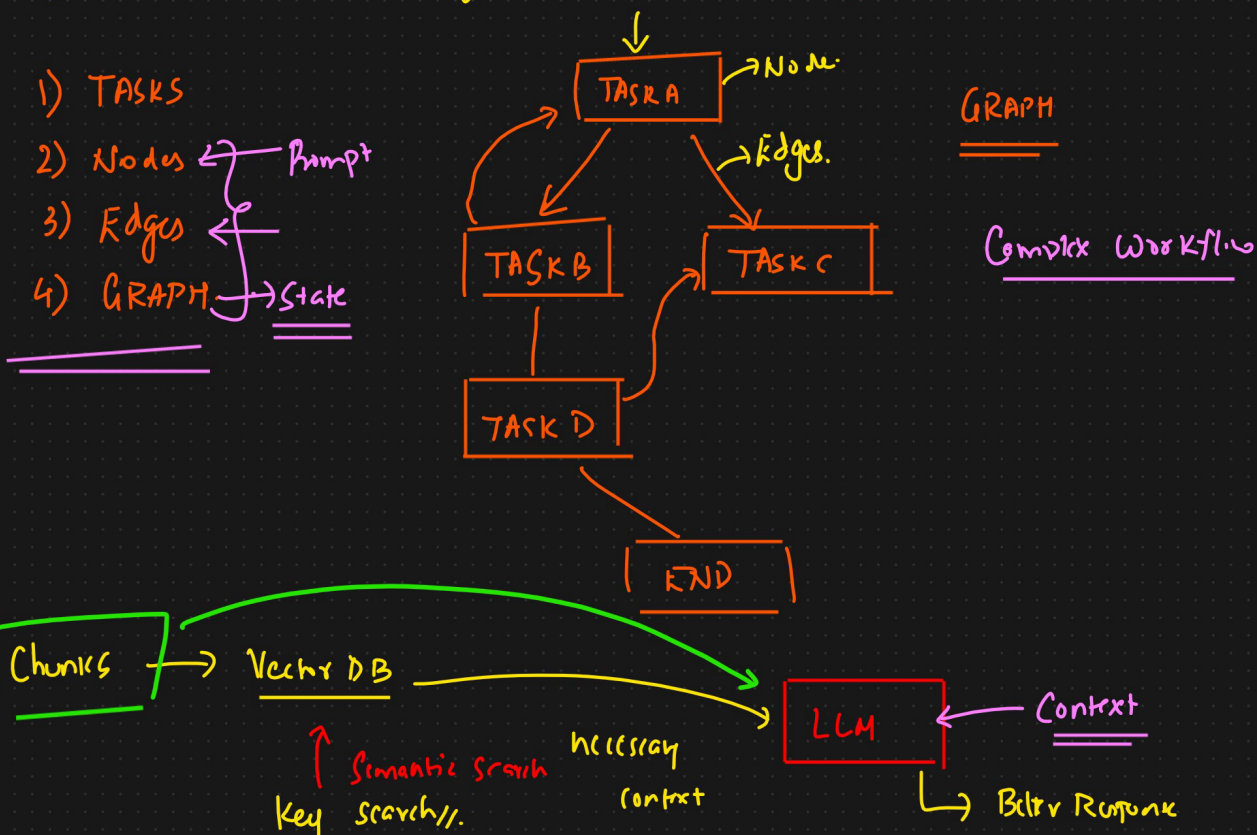
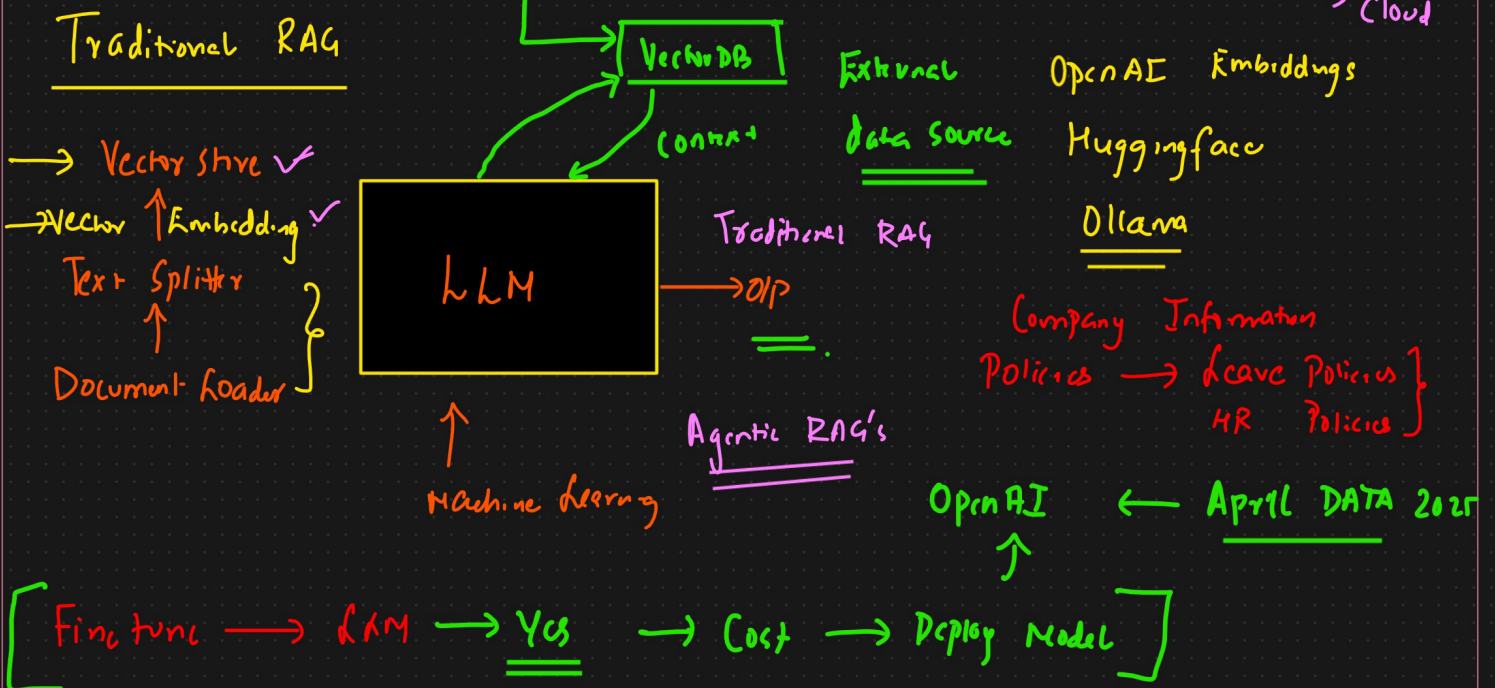
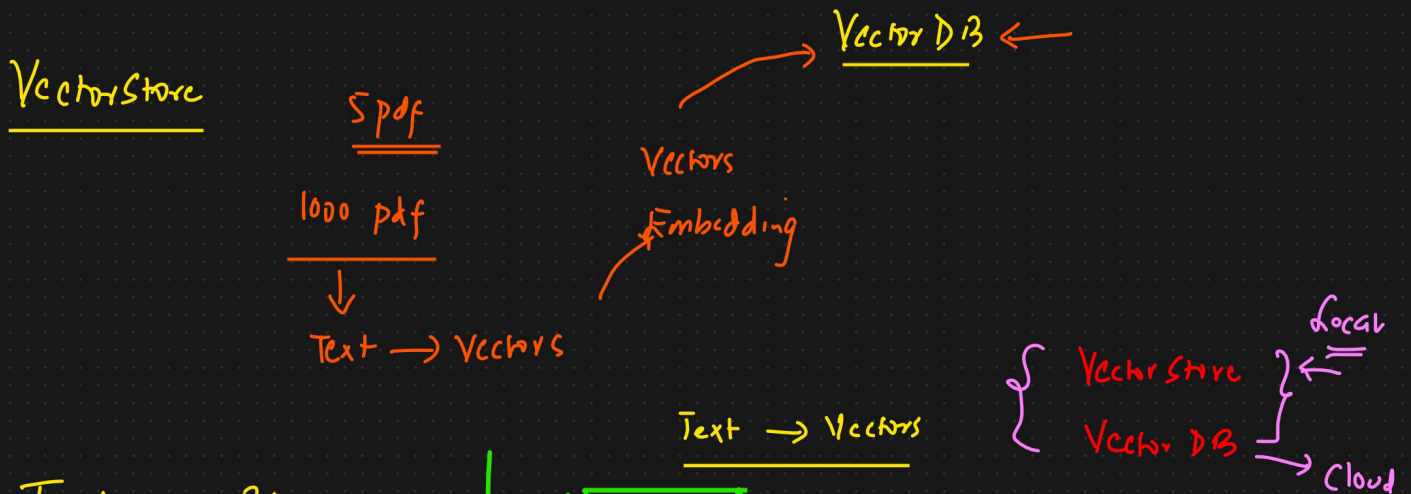
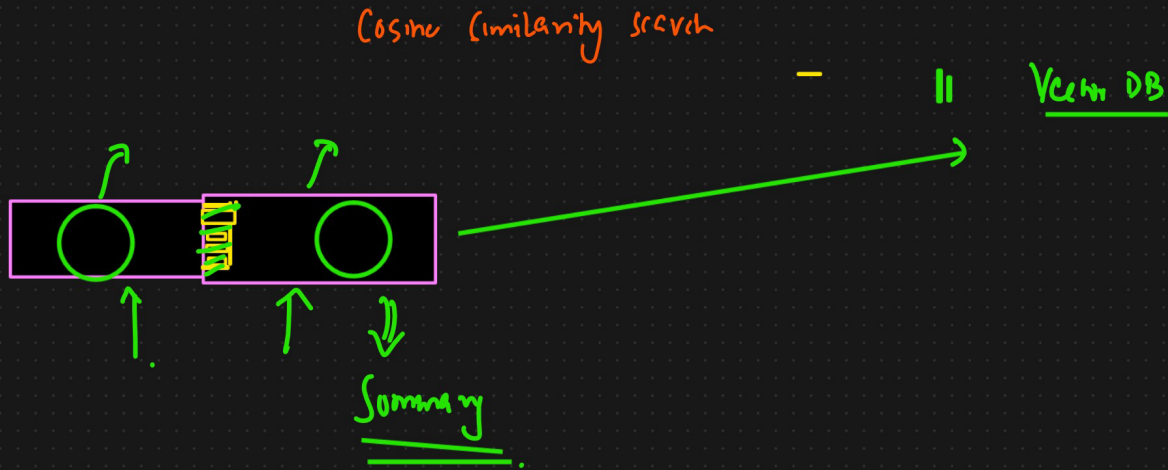
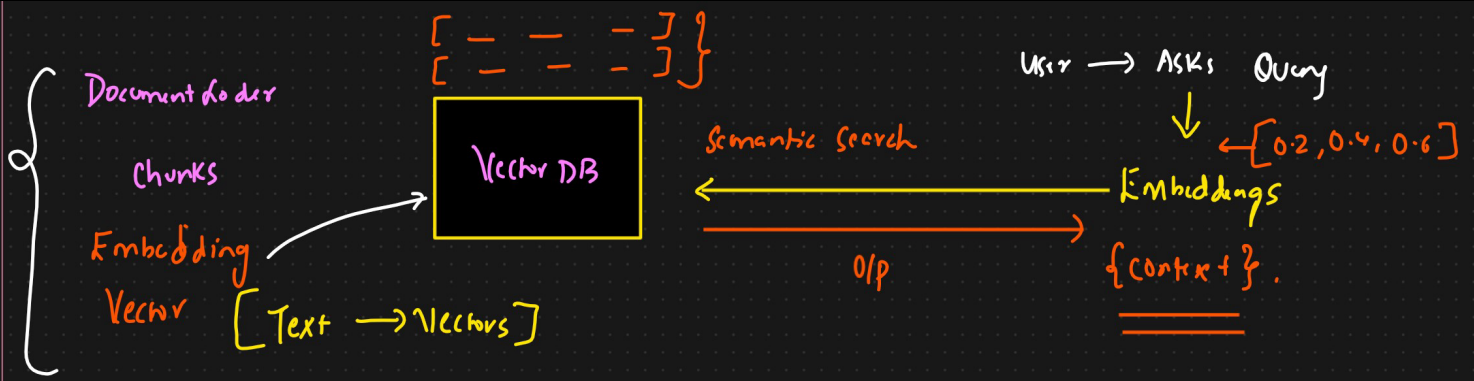


LANGCHAIN + RAG



Langgraph ÷ Stateful Multi AI Agents → Communicating → Complex Workflow.





RAG → Retrieval Augmented Generation.

Fine Tuning =
ChatBot ÷ Hi

Hi Welcome to KRISHNA ACADEMY

What would you like to learn?

GROK ←

Prompts → Prompt Template →

"Prompt" → System

"Fine Tuning" ⇒ Domain ← Company

"RAG" ⇒ External → keeps on changing

→ RAG
→ Tools
→ External API's

Imp.

★ Main Building Blocks in LangChain

Stage	Components / Purpose
Retrieve	- Document Loaders → bring data from PDFs, CSVs, web, DBs, etc. - Text Splitters → chunk text into smaller pieces (to fit context window). - Vector Stores → embed chunks + store them for semantic search.
Summarize	- Chains or agents that condense long documents into short, useful summaries. - Approaches: map-reduce, refine, stuff.
OP (Output Parsing)	- Output Parsers define the structure of LLM responses. - Examples: StrOutputParser, JsonOutputParser, PydanticOutputParser. - Ensures raw LLM text → structured data for your app.

why chunking

↳ Because there is a limit to Context Window

v.v. Imp.

◆ What LangChain gives you

LangChain is a **framework** — think of it as a *toolbox* to connect LLMs with data, prompts, workflows, and apps.

It provides building blocks like:

- **Document loaders** → load PDFs, CSVs, websites, DBs
- **Text splitters** → break big docs into chunks
- **Embeddings + Vector Stores** → semantic search
- **Chains/Agents** → orchestrate workflows
- **Output Parsers** → structure results

LangChain vs LangGraph

Aspect	LangChain (chain of Tasks)	LangGraph (agent workflows)
Focus	General LLM apps (RAG, chatbots, pipelines)	Agentic AI (stateful, multi-step agents)
Structure	Linear <u>chains</u> of prompts, tools, and models	<u>Graphs</u> of nodes & edges (state machines)
State	Stateless by default	Stateful by design
When to Use	- Quick RAG apps - Prompt chaining - Lightweight chatbots.	- Autonomous multi-step agents - Tool-using assistants - Applications needing reliability & state.

Q: what is difference b/w Stateless & Stateful?

Stateless

- The system does **not** remember past interactions.
- Each request is **independent**.
- If you ask the same question twice, it gives an answer without "knowing" what was asked before.

✓ Example:

A calculator app → every calculation is fresh:

```
2 + 2 = 4
5 + 5 = 10
```

It doesn't remember the previous calculation.

Stateful

- The system **remembers** past interactions (state) and uses them in the next step.
- Each new request depends on the **history / context**.

✓ Example:

A chat app with memory:

```
vbnet
Copy code

You: Who is Elon Musk?
AI: He is the CEO of Tesla and SpaceX.
You: Where was he born?
AI: Elon Musk was born in Pretoria, South Africa.
```

Here the AI remembers "he" refers to Elon Musk — that's being stateful.

Imp.

In your context (LangChain vs LangGraph):

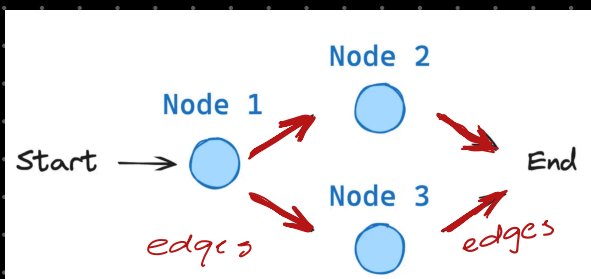
- LangChain → Stateless by default (you must add memory if you want the model to remember).
- LangGraph → Stateful by design (the graph naturally carries forward the agent's state across steps).

Lang Graph is based on the idea of DAG

DAG = Directed Acyclic Graph

↳ Graph of Nodes & edges

Blueprint
for
Lang Graph



Node = Each Task

Edge = Connects 2 Nodes

DAG = Entire process

Whole Agent = DAG

Components of Lang Graph

◆ 1. Each Node = a step → Each step (Task)

In LangGraph, a **node** is like a **block of work**.

Examples of nodes:

- **LLM Call Node** → calls GPT to generate text.
- **Tool Call Node** → fetches data from a database or API.
- **Decision Node** → chooses which path to follow (like "Do I need web search?").
- **Validation Node** → checks if output meets conditions (e.g., JSON format).

So every node is a **unit of logic**.

◆ 2. Each Edge = flow of control

→ Connects two Nodes Together

An **edge** connects one node to the next.

- If Node A finishes, an edge can point to Node B.
- Sometimes, **conditional edges** are used → if LLM says "Yes", go to Tool Node; if "No", go to Answer Node.

So edges decide **how information and execution moves**.

◆ 3. The whole Agent = a DAG

When you connect nodes with edges:

- The graph is **directed** (flows forward only).
- It's **acyclic** (no infinite loops).
- The **entire workflow/agent** is just this **DAG**.

Acyclic means no infinite loops.

◆ Example: Agent as DAG

less

```
[User Input] → [Decision Node: Need Tool?]  
  | Yes → [Tool Call Node] → [LLM Response Node] → [Output]  
  | No  → [LLM Response Node] → [Output]
```

- **Nodes:** Input, Decision, Tool, LLM, Output.
- **Edges:** connect them in sequence and conditionally.
- The **agent's logic** is nothing but this DAG.

COMPONENTS OF LAGCHAIN;

1. RETRIEVE

- Document loaders
- Text splitting
- Vector storage

2. SUMMARY

- chains
- context
- RAG
- memory

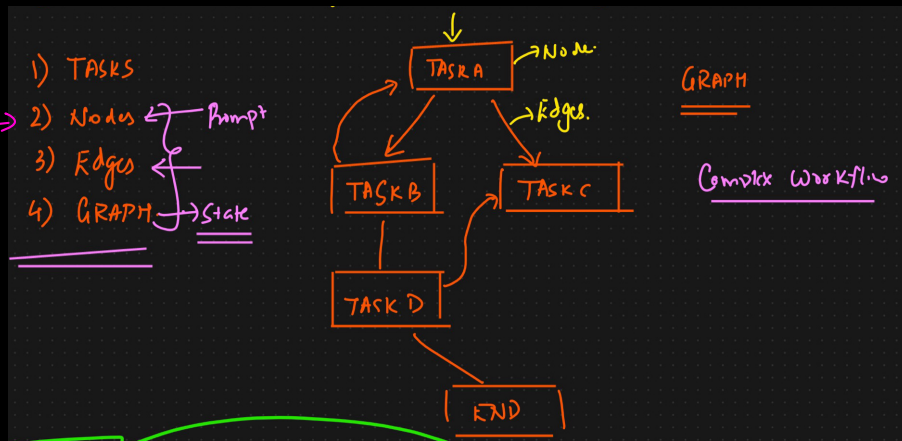
3. O/P

parsers

LANG GRAPH; Multi AI Agents / workflows

Nodes → Edges → o/p { Acyclic

each node will have prompts.



Imp. Memory is efficient in Lang Graph.

Practicals; LANGCHAIN

STEP I Document Loaders

- Text Loader = Loading Text
- Pdf Loader = PyPDF
- (bs4) • web base loader = Best Scraper
- Arxiv Loader = Research papers
- wiki Loader = Encyclopaedia.

```
from langchain_community.document_loaders.text import TextLoader
```

```
loader=TextLoader('speech.txt')
```

```
text_documents=loader.load()
```

```
text_documents
```

PyPdf

web base loader

Arxiv, wiki

STEP II Text Splitter break docs in chunks

most popular

- CharacterTextSplitter → Splits by fixed character length (simple, fast). e.g., 1000 characters.
- RecursiveCharacterTextSplitter → Splits smartly by paragraphs/sentences (best for most docs).
- TokenTextSplitter → Splits by tokens (respects LLM limits). e.g., 4k/8k context
- MarkdownTextSplitter → Splits by Markdown headings (good for structured docs).
- CodeTextSplitter → Splits by functions/classes (for code files).

Recursive Character Text Splitter; smart way to split text into chunks for embeddings / RAG ensuring we don't break meaning mid-sentence.

It's a text splitter in LangChain that breaks large documents into smaller chunks while preserving structure (paragraphs, sentences) as much as possible.

- Unlike a simple character splitter (which just cuts text every `n` characters),
- **RecursiveCharacterTextSplitter** tries to split intelligently:
 1. First by big sections (e.g., paragraphs).
 2. If still too long, then by sentences.
 3. If still too long, then by words.
 4. Finally, by characters if nothing else works.

➡ Basically, it "recursively" splits text until chunks fit into the size you want.

◆ Why is it Needed?

- Keeps chunks **semantically meaningful** (not cutting in the middle of a sentence).
- Ensures chunks stay within the **token limit** for embeddings / LLMs.
- Supports **chunk overlap** (a few words repeated between chunks) so context isn't lost.

◆ Example in Code

python

Copy code

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500, # max size of each chunk  
    chunk_overlap=50 # overlap between chunks  
)
```

```
long_text = """
```

```
LangChain is a framework for developing applications powered by language models...  
(Imagine this is a long document)  
"""
```

```
chunks = text_splitter.split_text(long_text)
```

—————→ 10 - 20% is always preferred.

Imp.

`\n\n` = paragraph

`\n` = sentence.

`" "` = word

`" "` = character

Imp. Cosine Similarity;

Cosine similarity measures the **angle** between two embedding vectors to determine how similar their meanings are, independent of their length. In a vector database, both documents and queries are converted into embeddings, and cosine similarity is used to find which vectors point in the most similar direction — those are returned as the most relevant results.

Formula:

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \|B\|}$$

How to split json

Recursive JSON Splitter.

Imp. Default character split = `"\n\n"`

Q1. Difference b/w Vector Store & Vector DB

Vector Store; is small scale (small datasets)

- mostly local setup
- example
 - FAISS → Local; similarity search
 - Chroma → local or cloud
 - Lightweight Embeddings

Vector Database; is production Ready

- scalable and large dataset
- pinecone → cloud based

Q1. What are Techniques for Embeddings

Accurate → **OpenAI Embedding** - Best & Paid
→ Hugging Face - Free Embedding
→ Ollama Embedding

Q1. What is LLM Fine Tuning?

Rewiring (Teaching) a model (LLM) to act like your company or Domain expert

- unlike RAG, the model's parameters are actually changed.

Example - Real World Scenario

Imagine Amazon Customer Support:

- The base LLM already knows general English.
- But Amazon wants the model to always:
 - Use polite, branded tone.
 - Follow exact refund/return policy rules.
 - Handle common scenarios consistently.

So Amazon collects thousands of chat transcripts and policy documents and fine-tunes the model.

Now, when a customer asks: "Can I return this item after 45 days?",

the model doesn't just answer generally—it replies in Amazon's tone and applies the specific return policy.

STEP II EMBEDDINGS

Conversion of text to vectors

- OpenAI embedding model = (Most Accurate)
- Hugging Face = (Open source Free)

Q1 Does Open AI use BOW or TFIDF Tech?

No, it uses transformer based embeddings where Feature Representation is the main character.

Code

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter=RecursiveCharacterTextSplitter(chunk_size=500,chunk_overlap=50)
final_documents=text_splitter.split_documents(docs)
final_documents

#Embedding the documents
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-large",dimensions=1024)

embeddings.embed_query(final_documents[0].page_content)
```

Imp: we can also use hugging face embedding

Hugging Face Coding

```
os.environ['HF_TOKEN']=os.getenv("HF_TOKEN")
✓ 0.0s

from langchain.embeddings import HuggingFaceEmbeddings

# Step 3: Initialize the embedding model
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

text="this is latest documents"
query_result=embeddings.embed_query(text)
query_result
✓ 7.2s
```

STEP IV VECTOR DATABASES vector store

- In Memory DB → RAM
 - On Disk DB → SSD, HDD
 - Cloud DB } AWS, Azure, Pinecone
- own server

FAISS → Facebook AI Similarity Search

Pinecone → Big Data

Types of Similarity Searches

1. Cosine Similarity

Angle b/w two vectors.

2. L2 Similarity

Euclidean Distance
Shortest Distance b/w two vectors

Types of Similarity Scores

Lets Check Similarity between Data and Query

Metric	Similarity Score Range	Behavior
Cosine Similarity	[-1, 1]	Focuses on angle only
L2 Distance	[0, ∞)	Focuses on magnitude + direction

```
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics.pairwise import euclidean_distances #L2
```

more are there

- Dot product
- Jaccard

Q. Why cant we use SQL / NO SQL DB?

- we need Similarity Search
- not exact text / Key word matches

Similarity Search

Because in RAG, you need to search by semantic similarity (meaning of text, via embeddings), not just exact matches or key-value lookups.

- SQL/NoSQL DBs → optimized for structured data, exact matches, filters, or keyword search.
- Vector DBs → optimized for **storing embeddings** and performing **nearest neighbor search** (cosine similarity, dot product, etc.) at scale, which is essential for retrieving the most relevant chunks of text for LLMs.

Also, Schema
SQL DB - Rigid
NoSQL DB - JSON

Q. What are important parts of vector DB?

- Index
- Similarity Search

INDEX, in vector DBs helps to make similarity search much easier by adding architectures like → HNSW and IVF

Index Types

Feature	Flat	IVF (Inverted File Index)	HNSW (Graph-based Index)
Type of Search	Exact	Approximate (cluster-based)	Approximate (graph-based traversal)
Speed	Slow (linear scan)	Fast (search only in top clusters)	Very Fast (graph walk)

Dataset Size	Recommended Index
UPTO 1L	IndexFlatL2 or IndexFlatIP
UPTO 1M	IndexIVFFlat or IndexHNSWFlat
> 1M	IndexIVFPQ or IndexHNSWFlat

Size

Vector Index Types – Comparison Table

Index Type	Definition	How It Works	Speed	Accuracy	Memory	Best Use Case
Flat (Brute-force) <i>Linear</i>	Stores all vectors and compares every one during search	Compares the query with every single vector using a similarity metric	❌ Slow	✅ Highest (100%)	✅ Low	Small datasets or when accuracy is critical
HNSW (Hierarchical Navigable Small World) <i>Graph</i>	A graph-based index where vectors are connected based on similarity	Builds a multi-layer graph; navigates it from top to bottom to find nearest neighbors quickly	✅ Fast	✅ High	❌ High	Large datasets, fast and accurate ANN search
IVF (Inverted File Index) <i>cluster</i>	Partitions the dataset into clusters, then searches within closest ones	Uses clustering (e.g., k-means) to group vectors; searches only in the nearest few clusters	✅ Fast	⚠️ Medium	✅ Medium	Very large datasets where memory efficiency matters

more recommended

Imp. In Flat we go thru each record but in HNSW and IVF, we don't.

Meta Data Data about Data (Author, Date Created)

STEP V: RETRIEVAL Search Engine

"Retrieval in RAG is like a search engine with an abstraction layer — it hides the internal mechanics (embeddings, similarity search, indexing) and just returns the most relevant results to be used by the LLM."

```
retriever=vector_store.as_retriever(
    search_kwargs={"k": 10} #hyperparameter
)
```

```
retriever.invoke("what is llama model?")
```

✓ 1.1s

Levels of Abstraction

1. **User level** → Just ask a question.
(You don't care *how* it finds the answer.)
2. **System level (hidden abstraction)** →
 - Embeds the query.
 - Compares against document embeddings.
 - Retrieves top-k matches.
3. **Output** → Most relevant documents (context)

Types of Retrievers

1. Lexical (Sparse) Retriever → TFIDF, Exact

- How it works: Matches documents using **exact words**.
- Techniques: TF-IDF, BM25, Elasticsearch.
- Example:
 - Query: "car price"
 - Document: "The car price is high" → ✓ matched
 - Document: "The automobile cost is high" → ✗ not matched (different words)

2. Dense (Vector) Retriever → Semantic Aware

- How it works: Converts query & documents into **embeddings** and finds the most **semantically similar**.
- Techniques: OpenAI embeddings, SentenceTransformers, FAISS, Pinecone.
- Example:
 - Query: "car price"
 - Document: "The automobile cost is high" → ✓ matched
 - Document: "The car model is new" → maybe less relevant (depends on semantic similarity)

3. Hybrid Retriever → Most preferred

- How it works: Combines **lexical + dense retrieval** to cover both exact keyword matches and semantic matches.
- Example:
 - Query: "side effects of Drug X"
 - Lexical match: Finds documents containing "Drug X" and "side effects"
 - Dense match: Finds documents mentioning "Drug X causes nausea" or "adverse reactions of Drug X"
 - Result: Merges both → higher recall and relevance

✓ Tip: In interviews, you can say:

"Lexical = exact word search, Dense = meaning-aware search, Hybrid = best of both worlds."

STEP VI ; GENERATION (Augment + LLM)

AUGMENTATION; Feeding the Context

◆ Retrieval

- What it does → Finds and fetches relevant information from a knowledge source.
- Analogy → Like a search engine returning the top 5 web pages.
- Output → A set of documents/passages (but not yet given to the LLM).

◆ Augmentation

- What it does → Takes the retrieved documents and injects (**augments**) them into the LLM's input prompt.
- Analogy → Like you take the top 5 Google results, copy the relevant paragraphs, and paste them into your essay draft before writing.
- Output → Enriched input prompt (user query + retrieved context).

◆ Key Distinction

- Retrieval = fetching knowledge.
- Augmentation = adding that knowledge to the model's prompt.

👉 Retrieval is about *finding*.

👉 Augmentation is about *supplying/feeding*.

User: "What are the side effects of drug X?"

1. Retrieval step →

Finds documents:

- Doc1: "Drug X may cause nausea..."
- Doc2: "In rare cases, headaches occur..."

2. Augmentation step →

Build the final LLM input:

diff

Context:

- Drug X may cause nausea
- Headaches occur in rare cases

Question:

What are the side effects of drug X?

GENERATION ; LLM

produces final Response using Retrieved knowledge

↳ Combination of Query + Retrieved context

✓ Key Point

- The LLM does not search the documents itself — it simply reads the augmented context and generates a fluent, coherent answer.
- Retrieval → gets the facts
- LLM → turns facts into human-readable answer

Set up RAG CHAIN

For Generation we need to set up RAG chain

- Context - (Retriever)
- PromptTemplate - Langchain Hub → Prompt Template
- Model - LLM
- OutPut Parser - StrOutputParser

Imp: $\text{Langchain} = \text{prompt} / \text{model} / \text{output parser}$
 $\text{Langchain} + \text{RAG} = \text{RAG} / \text{prompt} / \text{model} / \text{parser}$

PARTS OF RAG PIPELINE



✓ Summary

Phase	Role
Data Ingestion / Indexing	Prepare documents and embeddings for search
Retrieval	Fetch relevant information for the query
Generation <i>Augment</i>	Use LLM to generate grounded answers using retrieved context

Imp: Some advanced pipelines also include:

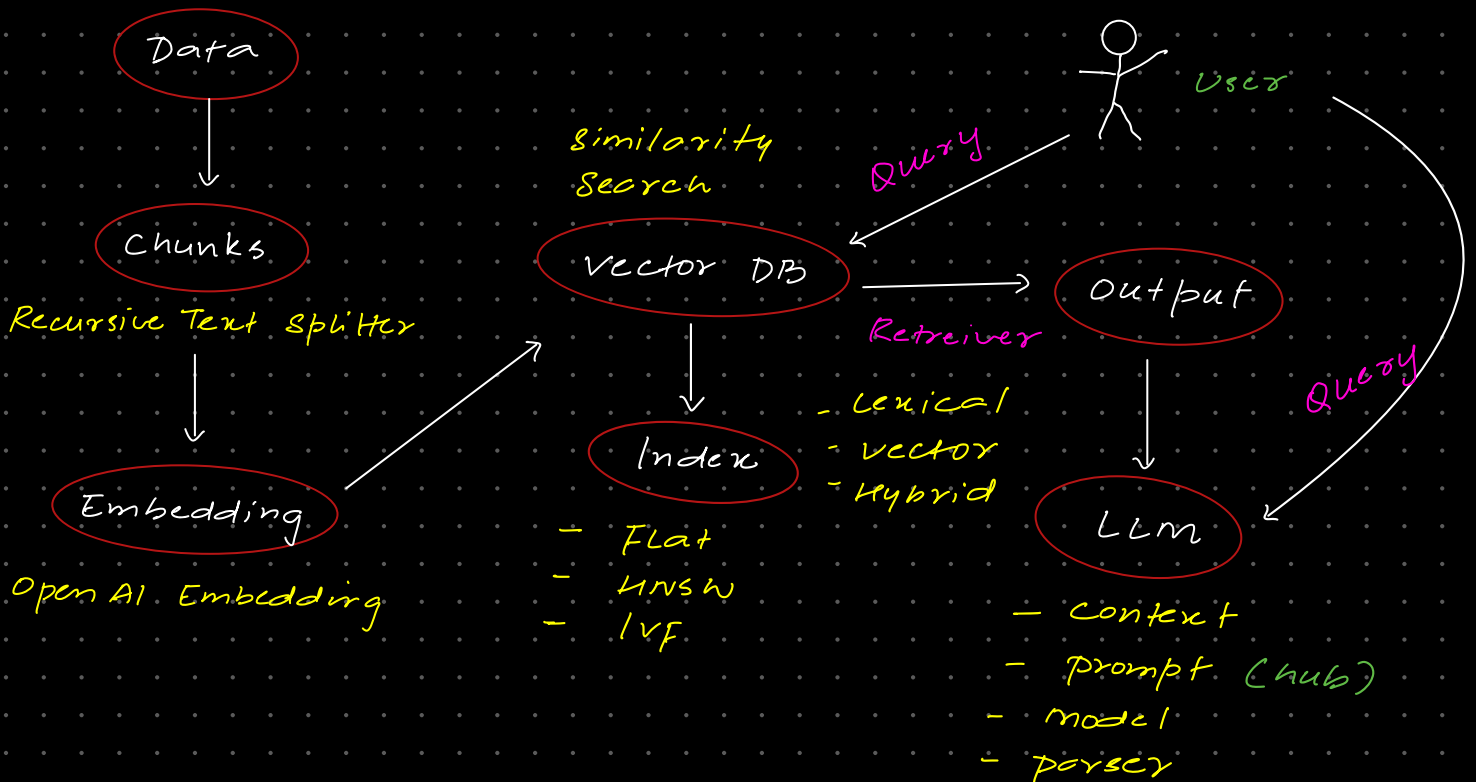
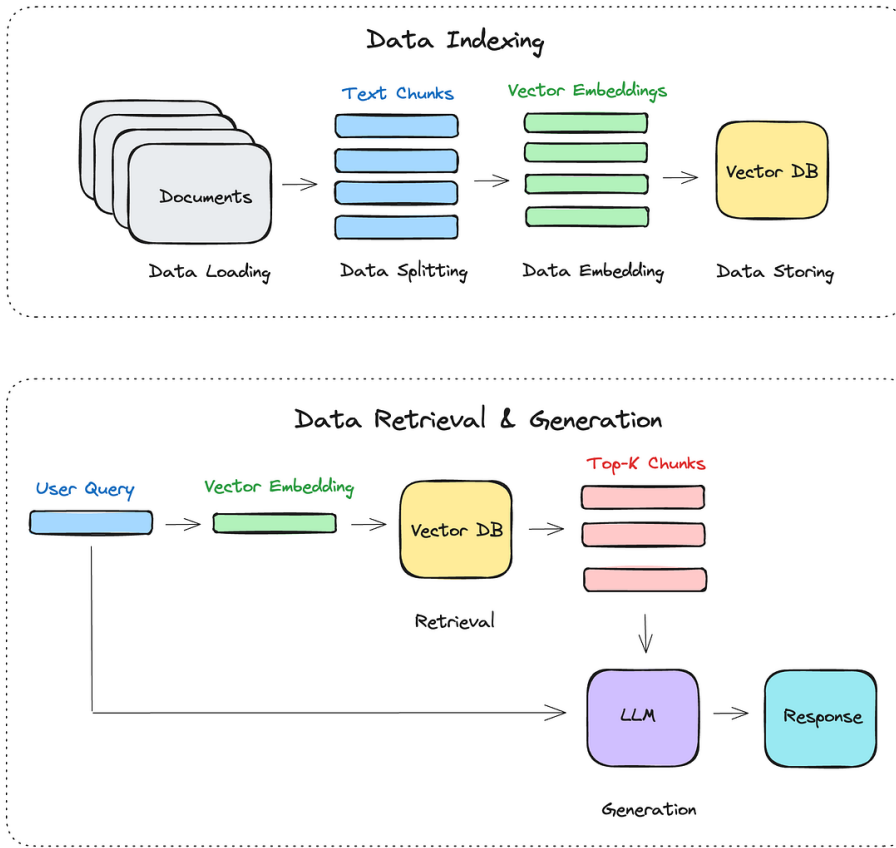
- Re-ranking (after retrieval)
- Multi-hop retrieval (chaining queries for reasoning)
- Caching / Memory (for repeated queries or session context)

→ Load, embed, vector DB

→ Augment + Generate.

LLM + RAG PIPELINE

Basic RAG Pipeline



Important points

Sparse Retrieval → TFIDF ; (Lexical)

Dense Retrieval → Embedding model

Hybrid Retrieval → Best of both worlds

Imp.

Q. How are contents saved in Vector DB and how are they retrieved?

Data is stored as

- page content
- meta data

Example:

css

Copy code

Retrieved result:

```
{
  content: "Drug X may cause nausea and headaches.",
  metadata: {source: "medical_doc.pdf", page: 12}
}
```

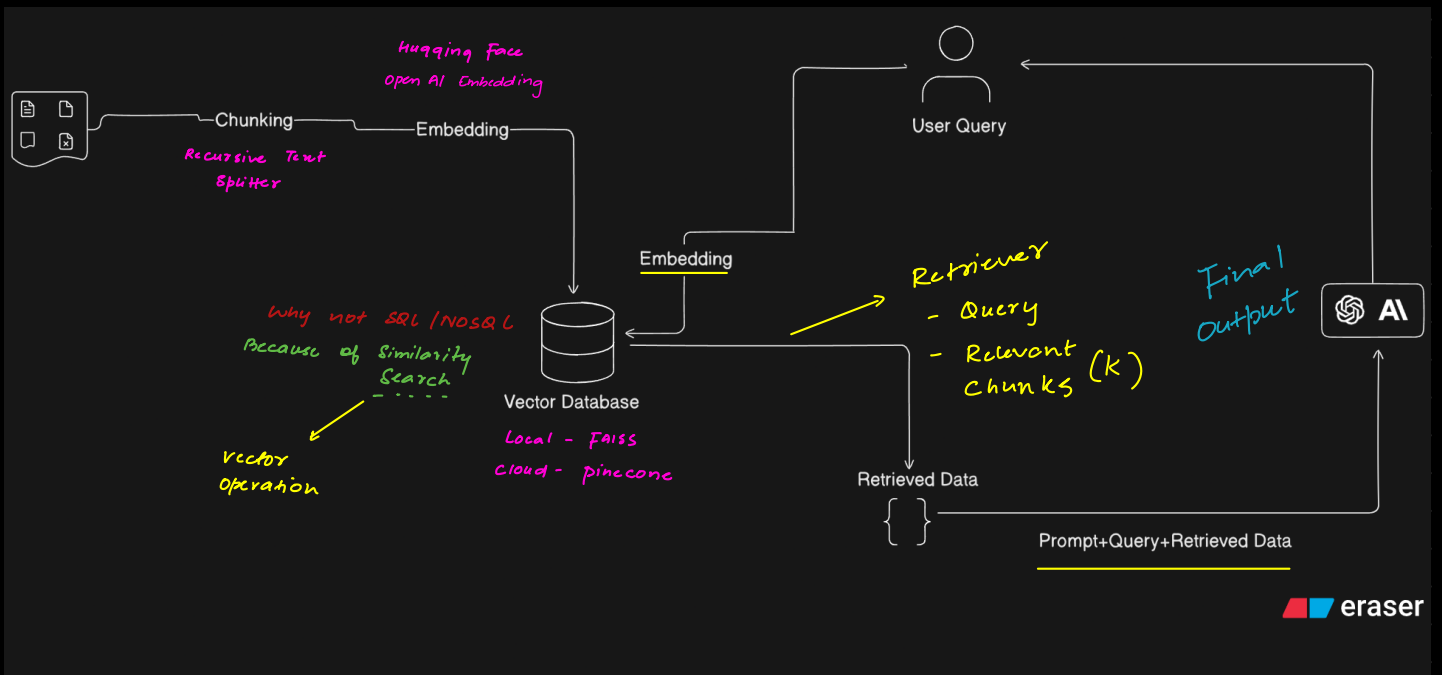
◆ Why Store Content + Metadata (Not Just Embeddings)?

- Content → needed to give context to the LLM.
- Metadata → useful for filtering (e.g., only show docs from 2023) and for **traceability** (so you can cite the source).

✓ Summary:

Yes — in a vector DB, documents are saved as embeddings + their raw content + metadata.

- Retrieval fetches these back (content + metadata) along with similarity scores.
- The embeddings alone are for search, but the **real text (content)** is always stored so the LLM can use it.



PINECONE

- FAISS is for small scale projects
- FAISS is for local (open source)

However;

pinecone offers fully cloud (SaaS.)

- Free version
- Enterprise version } Streamlit

2. Pinecone

- **Type:** Managed Vector Database (SaaS)
- **Where it runs:** Cloud (fully hosted, API-based)
- **Key Features:**
 - Stores embeddings + content + metadata out of the box.
 - Provides filtering, indexing, and scaling automatically.
 - Handles persistence, replication, and API queries.
 - Ideal for production-grade RAG systems.
- **Pros:**
 - ✓ Fully managed, no infra headaches
 - ✓ Metadata + filtering support built-in
 - ✓ Scales automatically (millions to billions of vectors)
- **Cons:**
 - ✗ Paid service (not free for large scale)
 - ✗ Less customizable than FAISS

Q11 why do we need cloud based ?

FAISS may be good for small scale or experiments but for production, we need; we need AWS, Azure or pinecone

Vendor	Local Support	Cloud Hosted
FAISS	Yes	No
Quadrant	Yes	Yes
Weaviate	Yes	Yes
Pinecone	No (API only)	Only
Milvus	Yes	(Zilliz)
Chroma	Yes	No

Q12 Types of indexes and "K" value

1. Exact

2. Approximate

Flat

IVF

HNSW

(cluster)

(Graph)

K = KNN
(fetched by)

Hyperparameter

fetched by
K = ANN

K Nearest
Neighbours

Approximate
Nearest Neighbour

Term	Full Form	Meaning in Simple Terms
KNN	K Nearest Neighbors	Exact search : Return top K truly closest points
ANN	Approximate Nearest Neighbors	Faster search : Return top K almost closest points (approximate)

Modality is form of Data a model can handle

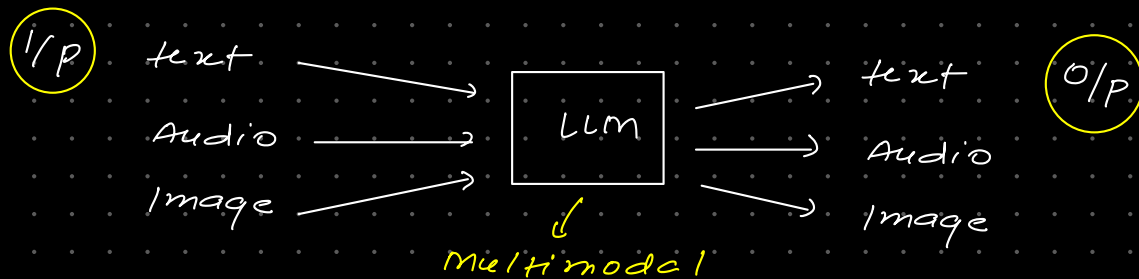
Multiple Modalities when we are combining or handling more than one Data type.

example =

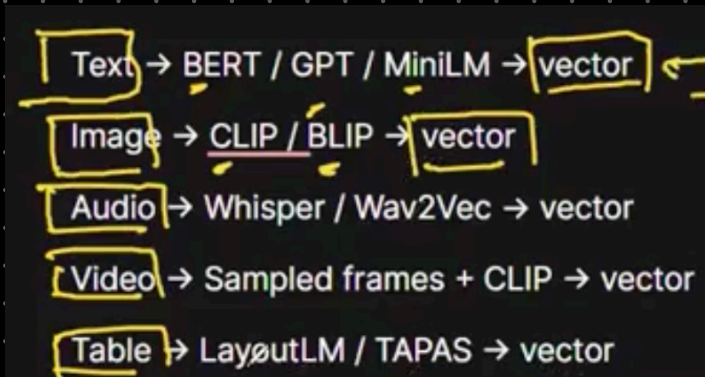
Example 1: Text + Image

- A multimodal model like GPT-4V or CLIP can look at an image and explain it in words.
- Query: "What's in this picture?"
- Model: "A cat sitting on a laptop."

Multimodal Data → text, image, audio, video



Qr. How to convert everything to vector?



Ultimately, data is stored in vectors only.

Imp. Weaviate can store images (Raw), text and metadata unlike pinecone.

AWS → OpenSearch built on top of Elastic Search.

Coding Example

Pinecone

```
{"id": "doc-1", "values": [0.22, 0.88, ...]}
```

Weaviate

```
{
  "class": "Product",
  "properties": {
    "image": "path.jpg",
    "text": "Best shirt in market"
  }
}
```

Vector DB	Verdict	Use-case Fit
Milvus (Zilliz)	good for multimodal	Image + Text + Video + Audio, production scale
<u>Qdrant</u>	Excellent, very flexible	Text + Image + Metadata hybrid search
Weaviate	Good with Transformers + hybrid	RAG, text + image + metadata combo
Pinecone	Text-only focus (no native image)	Great for text-based RAG, not for images
Vespa.ai	Enterprise-heavy multimodal <u>infra</u>	Multimodal + advanced query logic
AWS OpenSearch (KNN)	→ on top of elastic search	
Azure AI Search		
GCP Vertex AI Vector Search		

Imp.

Imp. while creating index mention dim length

Hugging Face - 384

Gemini - 768

Open AI - 1024

Imp.

pinecone is a wrapper over different cloud, it itself isn't a cloud.

uuid = universal unique I.D.

IMPORTANT POINTS

Imp. Faiss is offline
Imp. Pinecone is cloud based (SaaS).

Imp. very important

Always check

if embedding model = 768 Dimen.

→ Vector DB indexing should also be 768